

基于多因素评估的示例问题建模与分析

摘 要 本文以一个多因素决策问题为例，建立可复用的数据处理、综合评价与优化框架。首先对自主收集的数据进行缺失值检查和无量纲化，再构造综合评价函数，并通过权重扰动和留出样本检验结论的稳定性。结果表明，示例模型能在不同情景下给出一致排序，且关键权重小幅变化不会改变主要结论。最后给出模型局限与改进方向。完整可运行程序、自主查询数据说明及必要中间结果应作为支撑材料提交，并在附录中列出文件清单。

关键词 数学建模；综合评价；敏感性分析；优化

1 问题重述

1.1 问题背景

某管理部门需要依据多项效益型与成本型指标评价若干对象，并在预算受限条件下制定资源配置方案。原始数据具有量纲不同、个别记录缺失和指标重要性不一致等特点，因此需要建立可解释、可复现的分析流程。

1.2 问题提出

本文将任务概括为以下三个相互衔接的问题。固定宽度的问题标签可保证正文起点一致，长文本换行后仍与首行正文对齐。

问题1：对原始数据进行质量检查、缺失值处理和无量纲化，给出能够支撑后续建模的描述性结论；

问题2：建立综合评价与资源配置模型，说明权重和约束的来源，并给出可复现的求解结果；

问题3：从误差、参数扰动和情景变化三个角度检验结论，提出可解释的决策建议。

2 问题分析

2.1 问题一分析

问题一的核心是建立可靠的数据基础。先检查取值范围、重复记录和缺失比例，再根据指标含义选择中位数填补、截尾或保留缺失标记等方法，最后分别处理效益型和成本型指标。

2.2 问题二分析

问题二需要把多维指标映射为可解释的评价结果。可结合题意、专家判断或客观赋权确定权重，再通过线性加权、TOPSIS等方法计算综合得分，并把资源配置写成带预算和容量约束的优化问题。

2.3 问题三分析

问题三关注结论是否依赖某一组参数。可对权重、预算或观测误差进行扰动，比较排序、目标函数与关键决策变量的变化，并据此划定模型的稳定适用区间。

3 模型假设与符号说明

3.1 模型假设

1. 数据记录真实可靠，异常值处理不会改变总体分布的主要特征；
2. 研究期内指标含义和对象之间的基本关系保持稳定；
3. 未观测因素造成的影响可通过敏感性分析近似评估；
4. 各项资源可按模型给定的最小单位进行配置。

3.2 符号说明

表 1 主要符号说明

符号	含义
x_{ij}	第 i 个对象在第 j 个指标上的原始观测值
z_{ij}	无量纲化后的指标值
w_j	第 j 个指标的权重, 满足 $\sum_j w_j = 1$
S_i	第 i 个对象的综合得分
y_i	分配给第 i 个对象的资源量
B	可用资源总量

4 数据预处理与探索性分析

对效益型指标采用极差标准化:

$$z_{ij} = \frac{x_{ij} - \min_i x_{ij}}{\max_i x_{ij} - \min_i x_{ij}}. \quad (1)$$

成本型指标则将式 1 的分子改为 $\max_i x_{ij} - x_{ij}$ 。当某指标极差为零时, 将其标准化值统一记为零, 避免除零错误, 并在结果解释中说明该指标没有区分度。

探索性分析至少应报告样本量、缺失比例、极端值和主要相关关系。若使用网上自主查询的数据, 应记录来源、访问日期和清洗步骤, 并将实际使用的数据文件纳入支撑材料; 赛题提供的原始数据不需要重复提交。

5 问题一模型建立与求解

采用线性加权构造综合得分:

$$S_i = \sum_{j=1}^m w_j z_{ij}, \quad \sum_{j=1}^m w_j = 1, \quad w_j \geq 0. \quad (2)$$

权重应由题设、数据特征或有依据的方法确定。若改用 TOPSIS, 可参考相关经典方法 [1], 并完整说明正负理想解、距离和贴近度的计算过程。

表 2 给出可替换的示例结果。正式论文应报告实际数据、参数来源及不确定性, 而不是只给最终排名。

表 2 示例综合评价结果

对象	综合得分 S_i	排名	主要影响指标
A	0.842	1	指标 1
B	0.766	2	指标 2
C	0.693	3	指标 3

6 问题二模型建立与求解

在综合得分基础上，以总效益最大为目标建立资源配置模型：

$$\max \sum_{i=1}^n S_i y_i, \quad \text{s.t.} \quad \sum_{i=1}^n y_i \leq B, \quad 0 \leq y_i \leq u_i. \quad (3)$$

其中 u_i 为对象 i 的容量上限。若资源只能按整数单位分配，应补充 $y_i \in \mathbb{Z}$ ；若存在公平性、区域覆盖或风险约束，也应根据题意显式加入模型，而不是在求解后人为调整。

示例程序枚举小规模整数方案，便于模板直接运行。实际问题规模较大时，可替换为线性规划、整数规划或启发式算法，并报告求解器设置与终止条件。

7 问题三模型建立与求解

设基准权重为 w ，对第 j 个权重施加相对扰动 δ ，再将全部权重归一化。对每个扰动场景重新计算式 2 和式 3，记录排序变化率与目标函数相对变化：

$$R(\delta) = \frac{|F(\delta) - F(0)|}{|F(0)|}. \quad (4)$$

若在预设区间内排序不变且 $R(\delta)$ 较小，可认为主要结论对该参数具有稳定性。具体阈值应结合题目允许误差和实际决策成本说明，不能仅凭经验给出。

8 模型检验与敏感性分析

模型检验应覆盖数据、计算和结论三个层面：首先用留出样本或交叉验证评估外推误差；其次使用独立实现或极端输入核对程序；最后比较不同合理参数、标准化方式和情景假设下的主要结论。

对本示例而言，可将每个权重在基准值附近扰动 $\pm 10\%$ ，归一化后重新计算排名，并用问题三程序汇总各对象的最小、最大得分。若结论发生变化，应报告临界参数和受影响对象，而不应只展示稳定场景。

9 模型评价与结论

9.1 模型优点

模型的数据处理、评价、优化和检验链条清晰，各参数含义明确；程序与公式一一对应，便于复核和替换；敏感性分析能揭示结论对关键权重的依赖程度。

9.2 模型局限

线性加权假设可能忽略指标间的交互效应，权重来源也会影响排序。资源配置模型未描述所有现实约束时，其结果只能作为决策参考。后续可引入非线性效用、动态参数或更丰富的验证数据。

9.3 主要结论

本文给出了一套可复用的多因素评价与资源配置流程。正式作答时，应把本节替换为紧扣赛题的定量结论，说明适用条件、误差范围和实际意义，避免重复摘要或仅陈述求解步骤。

AI 工具使用声明

本参赛队在竞赛过程中未使用任何 AI 工具。

参考文献

- [1] Hwang, C. L., and Yoon, K. *Multiple Attribute Decision Making: Methods and Applications*. Springer, 1981.

A 支撑材料文件列表

- code/problem1.py、code/problem2.py、code/problem3.py: 完整可运行源程序;
- data/external-data.xlsx: 自主查询并实际使用的数据 (如有);
- results/intermediate-results.xlsx: 支撑结论的必要中间结果 (如有);
- AI 工具使用详情.pdf: 仅在真实使用 AI 工具时按官方规定提供。

B 完整源程序代码

```
1 """Problem 1: compute min-max normalized weighted scores."""
2
3
4 def normalize_columns(rows):
5     columns = list(zip(*rows))
6     normalized = []
7     for row in rows:
8         current = []
9         for value, column in zip(row, columns):
10             low, high = min(column), max(column)
11             current.append(0.0 if high == low else (value - low) / (high - low))
12         normalized.append(current)
13     return normalized
14
15
16 def weighted_scores(rows, weights):
17     if any(len(row) != len(weights) for row in rows):
18         raise ValueError("Each row must have the same length as weights")
19     return [sum(value * weight for value, weight in zip(row, weights)) for row in
20             rows]
21
22 if __name__ == "__main__":
23     data = [[8.0, 6.0, 7.0], [7.0, 9.0, 6.0], [6.0, 7.0, 9.0]]
24     result = weighted_scores(normalize_columns(data), [0.4, 0.35, 0.25])
25     print([round(value, 4) for value in result])
```

代码 1 问题一完整源程序

```

1 """Problem 2: solve a small integer resource-allocation example."""
2
3
4 def best_allocation(scores, capacities, budget):
5     if len(scores) != len(capacities):
6         raise ValueError("scores and capacities must have equal length")
7     best_value, best_plan = float("-inf"), None
8
9     def search(index, remaining, plan):
10         nonlocal best_value, best_plan
11         if index == len(scores):
12             value = sum(score * amount for score, amount in zip(scores, plan))
13             if value > best_value:
14                 best_value, best_plan = value, plan.copy()
15             return
16         for amount in range(min(capacities[index], remaining) + 1):
17             search(index + 1, remaining - amount, plan + [amount])
18
19     search(0, budget, [])
20     return best_plan, best_value
21
22
23 if __name__ == "__main__":
24     allocation, value = best_allocation([0.842, 0.766, 0.693], [3, 4, 5], 7)
25     print({"allocation": allocation, "objective": round(value, 4)})

```

代码 2 问题二完整源程序

```

1 """Problem 3: summarize score ranges under weight perturbations."""
2
3
4 def normalize_weights(weights):
5     total = sum(weights)
6     if total <= 0:
7         raise ValueError("The weight sum must be positive")
8     return [weight / total for weight in weights]
9
10
11 def score_ranges(normalized_data, base_weights, relative_change=0.1):
12     scenarios = []
13     for index in range(len(base_weights)):
14         for direction in (-1.0, 1.0):
15             changed = base_weights.copy()
16             changed[index] *= 1.0 + direction * relative_change

```

```

17         weights = normalize_weights(changed)
18         scenarios.append([
19             sum(value * weight for value, weight in zip(row, weights))
20             for row in normalized_data
21         ])
22     return [(min(values), max(values)) for values in zip(*scenarios)]
23
24
25 if __name__ == "__main__":
26     data = [[1.0, 0.2, 0.5], [0.5, 1.0, 0.0], [0.0, 0.4, 1.0]]
27     ranges = score_ranges(data, [0.4, 0.35, 0.25])
28     print([(round(low, 4), round(high, 4)) for low, high in ranges])

```

代码 3 问题三完整源程序